Deep Neural Network and Compression

Group 6

Group Leader:
Aditya Durgapal

Group Members:
Dev Kushawah
Priyanshu Mehta
Adish Kapate
M. Mithilesh
Shubham Joshi
Aditya Durgapal
Harsh Vardhan
Chaitanya Gaikwad
Nakshatra Kumar

January 7, 2026

Contents

1 Introduction to Image Representation **5**
1.1 Introduction . . . 5
1.1.1 Matrix Structure . . . 5
1.2 Development Environment and Loading . . . 5
1.3 Grayscale Processing . . . 5
1.3.1 Conversion Methods . . . 5
1.4 The Frequency Domain . . . 6
1.4.1 DFT vs. FFT . . . 6
1.4.2 Frequency Components . . . 6
1.4.3 Interpreting Frequencies . . . 6
1.5 Image Filtering in Frequency Domain . . . 6
1.5.1 Low Pass Filter (LPF) . . . 7
1.5.2 High Pass Filter (HPF) . . . 7
1.6 Implementation Workflow . . . 7

2 Image Filters and Color Spaces **8**
2.1 Histograms . . . 8
2.1.1 Grayscale Histogram . . . 8
2.1.2 RGB Histogram . . . 8
2.2 Color Spaces . . . 8
2.2.1 HSV (Hue, Saturation, Value) . . . 9
2.2.2 HSL (Hue, Saturation, Lightness) . . . 9
2.3 Basic Image Enhancements . . . 9
2.3.1 Brightness . . . 9
2.3.2 Contrast . . . 9
2.3.3 Saturation Adjustment . . . 9
2.3.4 Hue Shift . . . 10
2.4 Advanced Corrections . . . 10
2.4.1 Gamma Correction . . . 10
2.4.2 White Balance . . . 10
2.4.3 Vibrance vs. Saturation . . . 10

3 Convolution, Gradients, and Filters **11**
3.1 Convolution . . . 11
3.1.1 Mathematical Definition . . . 11
3.1.2 Properties of Kernels . . . 11
3.2 Blurring . . . 11
3.2.1 Average Blur . . . 12

3.2.2 Gaussian Blur . . . 12
3.3 Gradients and Differentiation . . . 12
3.3.1 Mathematical Formulation . . . 12
3.4 Edge Detection . . . 12
3.4.1 Sobel Operator . . . 12
3.4.2 Canny Edge Detection . . . 13
3.5 Sharpening . . . 13
3.5.1 Laplacian Sharpening . . . 13
3.5.2 Unsharp Masking . . . 13
3.6 Frequency Domain Analysis . . . 13

4 Feature Detection and Hough Transforms 14
4.1 Feature Detection: Corners . . . 14
4.2 Sobel/Canny vs. Hough Transform . . . 14
4.3 Hough Transform for Lines . . . 14
4.3.1 Concept and Duality . . . 14
4.3.2 Polar Representation . . . 15
4.3.3 Algorithm for Line Detection . . . 15
4.4 Circle Detection . . . 15
4.4.1 Fixed Radius . . . 15
4.4.2 Unknown Radius . . . 15
4.5 Ellipse Detection . . . 16
4.6 Generalized Hough Transform (GHT) . . . 16
4.6.1 Algorithm . . . 16
4.6.2 Handling Scale and Rotation . . . 16
4.7 Summary . . . 16

5 Project: Custom Data Pipeline for Fruits-360 17
5.1 Introduction . . . 17
5.2 Image Analysis and Frequency Domain . . . 17
5.2.1 Mathematical Representation . . . 17
5.2.2 Frequency Domain Analysis . . . 17
5.3 Feature Engineering: Color Statistics . . . 18
5.3.1 RGB vs. HSV Color Space . . . 18
5.3.2 Implementation: 6 Linear Color Features . . . 18
5.4 Feature Engineering: Shape Analysis . . . 18
5.4.1 Object Isolation via Thresholding . . . 18
5.4.2 Implementation: 6 Linear Shape Features . . . 18
5.5 Image Processing: Texture and Edges . . . 19
5.5.1 Local Binary Patterns (LBP) . . . 19
5.5.2 Canny Edge Detection . . . 20
5.6 Dataset Class Structure . . . 20
5.6.1 The `__getitem__` Method . . . 20
5.6.2 Output Format . . . 20

6 Mathematical Toolkit for Machine Learning **21**
6.1 Module 1: Linear Regression - The Mathematics of Prediction 21
6.1.1 Core Concept and Hypothesis 21
6.1.2 Objective Function: Mean Squared Error (MSE) 21
6.1.3 Optimization 21
6.1.4 Evaluation 22
6.2 Module 2: Logistic Regression - The Mathematics of Classification 22
6.2.1 Formulation 22
6.2.2 Objective Function: Binary Cross-Entropy (BCE) 22
6.2.3 Multi-Class Extension (Softmax) 23
6.2.4 Evaluation Metrics 23
6.3 Module 3: K-Means Clustering - The Mathematics of Grouping 23
6.3.1 Objective Function 23
6.3.2 Algorithm (Lloyd's Algorithm) 23
6.3.3 Initialization and Evaluation 24
6.4 Module 4: Multilayer Perceptron (MLP) - The Mathematics of Non-Linearity 24
6.4.1 Mathematical Formulation 24
6.4.2 Common Activation Functions 24
6.4.3 Backpropagation 24
6.5 Module 5: Convolutional Neural Networks (CNN) - The Mathematics of Perception 24
6.5.1 Core Building Blocks 25
6.5.2 The Convolution Operation 25
6.5.3 Pooling Operation 25
6.5.4 Key Principles 25

7 Model Generalization, Regularization and Neural Networks **26**
7.1 Model Generalization 26
7.2 Underfitting 26
7.2.1 Error Characteristics 26
7.3 Overfitting 26
7.3.1 Error Characteristics 26
7.4 Bias–Variance Trade-off 27
7.5 Regularization 27
7.5.1 L2 Regularization (Ridge) 27
7.5.2 L1 Regularization (Lasso) 27
7.6 Feature Scaling 27
7.7 Multilayer Perceptron (MLP) 27
7.8 Universal Approximation Theorem 28
7.9 Backpropagation 28

8 Convolutional Neural Networks and Model Compression **29**
8.1 Introduction to Convolutional Neural Networks 29
8.2 Limitations of Fully Connected Networks for Images 29
8.3 Convolution Operation 29
8.4 Weight Sharing and Translational Invariance 29
8.5 Activation Function 30
8.6 Pooling Operation 30

8.6.1 Max Pooling . . . 30
8.7 CNN Architecture . . . 30
8.8 Model Compression . . . 30
8.8.1 Pruning . . . 30
8.9 Quantization . . . 30
8.9.1 Memory Reduction . . . 31
8.10 Clustering-Based Quantization . . . 31
8.11 Gradient Flow in Quantized Networks . . . 31

9 Deep Learning Implementation with PyTorch 32
9.1 Introduction . . . 32
9.2 The Fundamental Unit: Tensors . . . 32
9.2.1 Creation and Operations . . . 32
9.3 Data Handling Pipeline . . . 33
9.4 Autograd: The Automatic Differentiation Engine . . . 33
9.4.1 Computational Graph . . . 33
9.4.2 Gradient Control . . . 33
9.5 The Training Workflow . . . 33

Lecture 1

Introduction to Image Representation

1.1 Introduction

In Computer Vision (CV), an image is defined as a visual representation of real objects (e.g., humans) or synthetic graphics. Mathematically, images are treated as matrices.

1.1.1 Matrix Structure

- A color image is represented as a matrix of dimensions $H \times W \times 3$, where H is height, W is width, and 3 represents the color channels.
- Each pixel is a tuple of (R, G, B) values ranging from 0 to 255 (uint8 format).
- For mathematical operations, these integer values are typically converted to `float32`.

1.2 Development Environment and Loading

The lecture recommends using Python libraries such as `opencv-python`, `numpy`, and `matplotlib` within a Jupyter Notebook or Google Colab environment.

Key Implementation Detail: Different libraries utilize different channel orders. OpenCV loads images in BGR format, whereas Matplotlib expects RGB. Therefore, it is strictly necessary to convert BGR to RGB for correct color visualization.

1.3 Grayscale Processing

Many computer vision pipelines, including edge detection, utilize grayscale images for simplicity. Converting to grayscale reduces the complexity of data while retaining structural information.

1.3.1 Conversion Methods

There are two primary methods to convert color to grayscale:

1. **Average Method:**

$$I_{GRAY} = \frac{R + G + B}{3} \tag{1.1}$$

   This simple average treats all channels equally.

2. **Weighted Method:**

$$I_{GRAY} = 0.299R + 0.587G + 0.114B \tag{1.2}$$

   This method accounts for human perception (we are more sensitive to green) and is the standard for edge detection.

1.4 The Frequency Domain

The lecture introduces the Fourier Transform (FT) as a method to break an image down into a sum of sinusoids of varying frequencies.

1.4.1 DFT vs. FFT

- **DFT (Discrete Fourier Transform):** The mathematical definition has a time complexity of $O(N^2)$, making it very slow for large images.
- **FFT (Fast Fourier Transform):** An optimized algorithm with complexity $O(N \log N)$, making it feasible for image processing.

1.4.2 Frequency Components

The output of an FFT includes:

- **Magnitude:** Represents how strong a frequency is (texture). To visualize this, a log scale is often used: $Mag = \log(|F_{(u,v)}| + 1)$.
- **Phase:** Represents where the frequency occurs (structure/edges). Phase is critical as it preserves the spatial structure and shapes of the image.

1.4.3 Interpreting Frequencies

- **Low Frequencies:** Correspond to smooth regions, backgrounds, and soft color transitions.
- **High Frequencies:** Correspond to edges, textures, noise, and fine details.

1.5 Image Filtering in Frequency Domain

Filtering involves modifying the frequency spectrum of an image using masks.

1.5.1 Low Pass Filter (LPF)

An LPF retains low-frequency components while removing high frequencies.

- **Mask:** White (1) in the center, Black (0) in the outer region.
- **Effect:** The resulting image is smoothed and blurred.

1.5.2 High Pass Filter (HPF)

An HPF retains high-frequency components while removing low frequencies.

- **Mask:** Black (0) in the center, White (1) in the outer region.
- **Effect:** The resulting image highlights edges, sharpens details, and extracts structure.

1.6 Implementation Workflow

The general workflow for applying frequency domain filters is as follows:

1. Convert image to float32.
2. Compute FFT using `np.fft.fft2`.
3. Shift zero-frequency components to the center using `np.fft.fftshift`.
4. Apply the specific mask (LPF or HPF) via multiplication.
5. Perform the inverse shift using `np.fft.ifftshift`.
6. Compute the Inverse FFT using `np.fft.ifft2` to return to the spatial domain.

Lecture 2

Image Filters and Color Spaces

2.1 Histograms

A histogram is a plot showing the frequency of pixels at each intensity value, typically ranging from 0 to 255. It provides a graphical representation of the tonal distribution in a digital image.

2.1.1 Grayscale Histogram

The X-axis represents intensity (0 = black, 255 = white), and the Y-axis represents the pixel count. The shape of the histogram indicates the exposure and contrast of the image:

- **Leaning Left:** The image is dark (underexposed).
- **Leaning Right:** The image is bright (overexposed).
- **Centered:** The image has balanced exposure.
- **Narrow Width:** The image has low contrast (pixels are clustered in a small range).
- **Wide Width:** The image has high contrast (pixels span the full range).

2.1.2 RGB Histogram

While grayscale histograms show overall brightness, RGB histograms plot separate distributions for the Red, Green, and Blue channels.

- **High Red values:** Indicates a warm image.
- **High Blue values:** Indicates a cold image.
- **Unequal Peaks:** Suggests a color imbalance or color cast.

2.2 Color Spaces

Different color spaces allow us to manipulate images based on how humans perceive color rather than just raw pixel values.

2.2.1 HSV (Hue, Saturation, Value)

HSV is a perception-based model:

- **Hue (H):** The "pure" color, represented as an angle on the color wheel. (In OpenCV, this is scaled to 0-180).
- **Saturation (S):** The vividness or purity of the color. Lower saturation approaches grayscale.
- **Value (V):** The brightness or lightness of the pixel.

2.2.2 HSL (Hue, Saturation, Lightness)

HSL is similar to HSV but differs in how it handles lightness:

- $L = 0$ is pure black, $L = 1$ is pure white.
- True color exists at $L = 0.5$.
- Unlike HSV, saturation in HSL is strongest at $L = 0.5$ and weak at the extremes (black or white).

2.3 Basic Image Enhancements

2.3.1 Brightness

Brightness refers to the overall luminance of pixels. It is adjusted by adding a constant bias (β) to the pixel values.

$$\text{Pixel}_{new} = \text{Pixel}_{old} + \beta \tag{2.1}$$

- Increasing β shifts the histogram to the right.
- **Note:** This does not increase contrast; it simply washes out the image if pushed too far (clipping).

2.3.2 Contrast

Contrast defines the difference in brightness between light and dark areas. It is modified by scaling the pixel values by a factor (α).

$$\text{Pixel}_{new} = \alpha \times \text{Pixel}_{old} \tag{2.2}$$

- $\alpha > 1$: Increases contrast (stretches the histogram width).
- $\alpha < 1$: Decreases contrast (compresses the histogram width).

2.3.3 Saturation Adjustment

Saturation makes colors appear more vibrant. To adjust it programmatically: 1. Convert the image from RGB to HSV. 2. Scale the Saturation (S) channel. 3. Clip values to stay within the [0, 255] range. 4. Convert back to RGB.

2.3.4 Hue Shift

Hue manipulation rotates the color on the color wheel. For example, a shift of +120° typically swaps RGB mapping (e.g., Red becomes Green). This transformation changes color identity while preserving brightness and texture.

2.4 Advanced Corrections

2.4.1 Gamma Correction

Gamma (γ) is a non-linear operation used to align digital values with human vision. Human eyes are more sensitive to differences in dark tones than bright tones.

$$\text{Monitor Output} = (\text{Image Input})^{1/\gamma} \tag{2.3}$$

Standard monitors historically used $\gamma \approx 2.2$. Gamma correction allows us to brighten dark shadows without washing out bright highlights.

2.4.2 White Balance

White balance corrects color casts so that white objects appear truly white.

- **Gray World Assumption:** This algorithm assumes that the average color of a scene should be neutral gray (Average Red = Average Green = Average Blue).
- **Algorithm:** Calculate the average of each channel, compute a global average, and scale each channel so its average matches the global average.

2.4.3 Vibrance vs. Saturation

There is a distinct difference between these two:

- **Saturation:** Boosts *all* pixels equally. This can turn skin tones orange and unnatural.
- **Vibrance:** Also known as "smart saturation." It boosts pixels that currently have *low* saturation while leaving already saturated pixels alone. It is preferred for portraits as it protects skin tones.

Mathematical concept for Vibrance:

$$S' = S + \alpha \cdot (1 - S) \cdot S \tag{2.4}$$

This formula ensures that the boost factor (α) has less effect as S approaches 1 (already saturated).

Lecture 3

Convolution, Gradients, and Filters

3.1 Convolution

Convolution is the fundamental mathematical operation behind most image processing filters.

3.1.1 Mathematical Definition

For an image I and a kernel K (typically a small matrix, e.g., 3×3), the convolution operation is defined as:

$$(I * K)(x, y) = \sum_i \sum_j I(x + i, y + j) \cdot K(i, j) \tag{3.1}$$

where $i, j \in \{-1, 0, 1\}$ for a 3×3 kernel.

3.1.2 Properties of Kernels

The effect of a convolution depends heavily on the weights in the kernel:

- **Sum of Weights:**
  - **Blur Kernels:** Usually sum to 1 (preserves brightness).
  - **Sharpen Kernels:** Sum > 1 or < 0.
  - **Edge Detectors:** Sum to 0 (zero-sum kernels).
- **Symmetry:** Gaussian blur kernels are symmetric, while Sobel kernels are directional.

3.2 Blurring

Blurring smooths the image by reducing high-frequency noise.

3.2.1 Average Blur

All pixels in the kernel have equal weight.

$$K = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Effect: Reduces noise but makes the image look soft and preserves no details.

3.2.2 Gaussian Blur

Weights decrease from the center, following a Gaussian distribution.

$$K = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

The mathematical function is:

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \tag{3.2}$$

Effect: Provides a more natural blur and preserves edges better than average blur.

3.3 Gradients and Differentiation

A gradient measures how fast pixel intensity changes and in which direction.

- **Slow change:** Small gradient.
- **Fast change (Edge):** Large gradient.

3.3.1 Mathematical Formulation

In 2D, we measure change in both x and y directions (G_x and G_y).

$$\text{Magnitude } S = ||\nabla I|| = \sqrt{\left(\frac{\partial I}{\partial x}\right)^2 + \left(\frac{\partial I}{\partial y}\right)^2} \tag{3.3}$$

$$\text{Direction } \theta = \tan^{-1}\left(\frac{\partial I/\partial y}{\partial I/\partial x}\right) \tag{3.4}$$

3.4 Edge Detection

3.4.1 Sobel Operator

Sobel uses two specific kernels to calculate derivatives in x and y directions:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

The final edge magnitude is $G = \sqrt{G_x^2 + G_y^2}$.

3.4.2 Canny Edge Detection

Canny is considered the "gold standard" for classical edge detection. It involves a multi-stage process:

1. **Gaussian Blur:** To remove noise.
2. **Sobel Gradients:** To find intensity gradients.
3. **Laplacian / Gradient Magnitude:** To highlight regions of rapid change.
4. **Non-max Suppression:** To thin out edges.
5. **Double Thresholding:** To classify strong and weak edges.
6. **Edge Tracking by Hysteresis:** To connect weak edges to strong edges.

3.5 Sharpening

3.5.1 Laplacian Sharpening

The Laplacian is a second-order derivative operator that detects regions where edges change quickly.

$$\text{Sharpened} = \text{Original} - \alpha \times \text{Laplacian}$$

It highlights points and fine details, making the image look crisper.

3.5.2 Unsharp Masking

This technique enhances sharpness by subtracting a blurred version of the image from the original.

1. Blur the image to get B.
2. Compute the mask $M = I - B$.
3. Add the mask back to the original:

$$I_{sharp} = I + kM \tag{3.5}$$

   where k controls the amount of sharpness.

3.6 Frequency Domain Analysis

Filtering can also be understood in the frequency domain (using FFT):

- **Gaussian Blur:** Acts as a low-pass filter (attenuates high frequencies). The FFT of a Gaussian kernel is a circular low-pass pattern.
- **Sharpening:** Acts as a high-pass filter (amplifies high frequencies). The FFT of a sharpening kernel is a ring-shaped high-pass pattern.

Lecture 4

Feature Detection and Hough Transforms

4.1 Feature Detection: Corners

To detect features in an image, we analyze a small patch around a pixel and observe how the intensity changes when the patch is shifted slightly.

- **Flat Region:** The patch looks the same in all directions. There is no significant intensity change.
- **Edge Region:** The patch looks the same along the edge direction but changes when moved perpendicular to the edge.
- **Corner Region:** The patch looks different in all directions. This uniqueness makes corners ideal features for tracking.

4.2 Sobel/Canny vs. Hough Transform

Traditional edge detectors (like Sobel and Canny) and the Hough Transform approach image analysis differently:

- **Sobel / Canny:** These rely on local gradient behavior. They do not consider prior information about the object's shape or structure.
- **Hough Transform:** This method considers the shape of the object as prior information. It maps the image into a parameter space to find instances of a parameterized shape (like a line or circle).

4.3 Hough Transform for Lines

4.3.1 Concept and Duality

The Hough Transform relies on a duality between the Image Space (x, y) and the Parameter Space (m, c).

- A line in the image space $y = mx + c$ corresponds to a point (m, c) in the parameter space.
- A point (x_i, y_i) in the image space corresponds to a line in the parameter space defined by $c = -x_i m + y_i$.

Multiple points lying on the same line in the image will generate lines in the parameter space that all intersect at a single point (m, c).

4.3.2 Polar Representation

Using the slope-intercept form ($y = mx + c$) is problematic because vertical lines have infinite slope ($m \to \infty$), requiring infinite memory. To solve this, we use the polar representation:

$$x \sin\theta - y \cos\theta + \rho = 0 \tag{4.1}$$

Where:

- θ is the angle of the normal, bounded by $0 \leq \theta < \pi$.
- ρ is the finite perpendicular distance from the origin.

4.3.3 Algorithm for Line Detection

1. Quantize the parameter space (θ, ρ).
2. Create an accumulator array $A(\theta, \rho)$ and initialize to 0.
3. For each edge point (x_i, y_i) in the image:
   - For each θ, calculate $\rho = x_i \sin\theta - y_i \cos\theta$.
   - Voting: $A(\theta, \rho) \leftarrow A(\theta, \rho) + 1$.
4. Find local maxima in $A(\theta, \rho)$ to identify lines.

4.4 Circle Detection

4.4.1 Fixed Radius

If the radius r is known, the problem reduces to finding the center (a, b). The equation of a circle is:

$$(x_i - a)^2 + (y_i - b)^2 = r^2 \tag{4.2}$$

Points in the image space vote for a center in a 2D parameter space. The intersection of these votes indicates the center.

4.4.2 Unknown Radius

If r is unknown, the accumulator array must be 3-dimensional: $A(a, b, r)$. This increases computational complexity exponentially. To optimize, we can use gradient information ($\nabla I(x, y)$). Knowing the gradient direction at an edge point restricts the possible center locations to a line, significantly reducing the search space.

4.5 Ellipse Detection

Ellipse detection requires a 5-dimensional parameter space, making it computationally expensive. The parameters are: Center (C_x, C_y), Major axis (R_x), Minor axis (R_y), and Orientation (θ).

The general equation used is:

$$\frac{((X - C_x)\cos\theta + (Y - C_y)\sin\theta)^2}{R_x^2} + \frac{((X - C_x)\sin\theta - (Y - C_y)\cos\theta)^2}{R_y^2} = 1 \tag{4.3}$$

4.6 Generalized Hough Transform (GHT)

For arbitrary shapes that cannot be described by a simple equation (e.g., a specific logo), we use the GHT. We define a reference point (x_c, y_c) relative to the object.

4.6.1 Algorithm

1. **Pre-processing:** Build an R-table mapping gradient direction ϕ_i to vectors $\vec{r} = (r, \alpha)$ pointing to the reference center.
2. Initialize accumulator $A(x_c, y_c) = 0$.
3. For each edge point (x_i, y_i) with gradient ϕ_i:
   - Retrieve all $\vec{r}$ entries for ϕ_i from the R-table.
   - For each corresponding $\vec{r}(r, \alpha)$:
     - $x_c = x_i \pm r\cos(\alpha)$
     - $y_c = y_i \pm r\sin(\alpha)$
     - $A(x_c, y_c) \leftarrow A(x_c, y_c) + 1$
4. Find maxima in $A(x_c, y_c)$ to locate the object.

4.6.2 Handling Scale and Rotation

If the object is scaled by s and rotated by θ, the accumulator becomes 4-dimensional $A(x_c, y_c, s, \theta)$. The update equations become:

$$x_c = x_i \pm r \cdot s\cos(\alpha + \theta) \tag{4.4}$$

$$y_c = y_i \pm r \cdot s\sin(\alpha + \theta) \tag{4.5}$$

4.7 Summary

- **Pros:** Works well on disconnected edges; insensitive to occlusion and noise.
- **Cons:** Efficient for simple shapes but impractical for complex shapes due to memory requirements (large accumulator arrays).

Lecture 5

Project: Custom Data Pipeline for Fruits-360

5.1 Introduction

This report documents the theoretical frameworks of Computer Vision (CV) and their practical application in our group project. The core of our implementation was the development of a custom **PyTorch Data Loader** for the Fruits-360 dataset. Rather than relying solely on raw pixel data, we engineered a feature extraction pipeline that isolates the region of interest using thresholding and computes a vector of mathematical invariants (Color and Shape) alongside texture and edge maps.

5.2 Image Analysis and Frequency Domain

5.2.1 Mathematical Representation

Digital images are represented as discrete matrices $I(x, y)$. In our pipeline, we handle images as multidimensional tensors. The raw data typically exists in RGB space, where a pixel is a vector $v \in \mathbb{R}^3$. However, for processing, we often convert to Grayscale $I_{gray} \in \mathbb{R}^{H \times W}$ to reduce dimensionality while preserving structural information.

5.2.2 Frequency Domain Analysis

The Discrete Fourier Transform (DFT) allows us to analyze the rate of change in pixel intensity. High frequencies correspond to edges and noise, while low frequencies correspond to smooth regions.

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} I(x, y) e^{-j2\pi(ux/M + vy/N)} \tag{5.1}$$

Operations such as High Pass Filtering (HPF) and Low Pass Filtering (LPF) are equivalent to masking specific regions in this frequency domain.

5.3 Feature Engineering: Color Statistics

5.3.1 RGB vs. HSV Color Space

While images are loaded in BGR/RGB format by OpenCV, this space is not ideal for feature extraction because intensity is correlated with color. We utilized the **HSV (Hue, Saturation, Value)** color space.

- **Hue:** Encodes the color information (wavelength).
- **Saturation:** Encodes the "purity" of the color.
- **Value:** Encodes the brightness.

5.3.2 Implementation: 6 Linear Color Features

In our codebase, the function `extract_color_features` performs the following operations: 1. Convert the input BGR image to HSV using `cv2.cvtColor`. 2. Compute the **Mean** (μ) and **Standard Deviation** (σ) for all three channels using `cv2.meanStdDev`. 3. Flatten these results into a single feature vector:

$$F_{color} = [\mu_H, \mu_S, \mu_V, \sigma_H, \sigma_S, \sigma_V] \in \mathbb{R}^6 \tag{5.2}$$

This statistical approach provides a global description of the fruit's appearance, robust to minor pixel variations.

5.4 Feature Engineering: Shape Analysis

5.4.1 Object Isolation via Thresholding

To analyze the shape of the fruit effectively, we first needed to segment it from the background. The Fruits-360 dataset has a white background (≈ 255). In our `extract_shape_features` function, we implemented an inverse binary threshold:

$$M(x, y) = \begin{cases} 1 & \text{if } I_{gray}(x, y) < 250 \\ 0 & \text{otherwise} \end{cases} \tag{5.3}$$

This creates a clean binary mask where the fruit is white (1) and the background is black (0). We then found the largest contour in this mask to represent the fruit boundary.

5.4.2 Implementation: 6 Linear Shape Features

From the extracted contour, we computed 6 specific scalar features in our code:

1. Area Ratio

To measure the relative size of the fruit within the 100×100 frame:

$$F_1 = \frac{\text{Contour Area}}{\text{Image Width} \times \text{Image Height}} \tag{5.4}$$

2. Aspect Ratio

Using the bounding rectangle (x, y, w, h), we calculated the ratio of width to height. This helps distinguish elongated fruits (like bananas) from round ones:

$$F_2 = \frac{w}{h} \tag{5.5}$$

3. Solidity

Solidity measures the convexity of the shape. It is the ratio of the contour area to the area of its convex hull (the "elastic band" shape around the object):

$$F_3 = \frac{\text{Contour Area}}{\text{Convex Hull Area}} \tag{5.6}$$

4. Circularity

This metric determines how close the shape is to a perfect circle. Based on the Isoperimetric Inequality:

$$F_4 = \frac{4\pi \times \text{Area}}{\text{Perimeter}^2} \tag{5.7}$$

5 & 6. Hu Moments (Log Transformed)

To ensure rotation and scale invariance, we calculated Image Moments. Specifically, we extracted the first two **Hu Moments** using `cv2.HuMoments`. Since raw Hu moments have a very large dynamic range, our implementation applies a logarithmic transformation:

$$F_{5,6} = -\text{sgn}(h_i) \cdot \log_{10}(|h_i| + 1e^{-12}) \quad \text{for } i \in \{1, 2\} \tag{5.8}$$

5.5 Image Processing: Texture and Edges

In addition to scalar features, our `get_processed_images` function generates two auxiliary image channels for every input.

5.5.1 Local Binary Patterns (LBP)

Texture provides critical surface information. We implemented LBP using efficient bitwise operations rather than loops.

1. We defined 8 spatial neighbors (top-left, top, top-right, etc.).
2. For each neighbor, we created a shifted version of the image using `np.roll`.
3. We compared the shifted image to the central image. If the neighbor pixel $\geq$ center pixel, the bit is set to 1.
4. These bits are shifted and combined using bitwise OR ($| =$) to form the final LBP pixel value.

$$LBP = \sum_{i=0}^{7} (I_{shift_i} \geq I_{center}) \cdot 2^{7-i} \tag{5.9}$$

5.5.2 Canny Edge Detection

To capture the structural skeleton of the fruit, we applied the Canny Edge Detector. Our implementation uses specific hysteresis thresholds determined experimentally:

```
canny = cv2.Canny(gray, 100, 200)
```

This explicitly extracts high-frequency gradients (edges) while suppressing noise via the internal Gaussian blur of the Canny algorithm.

5.6 Dataset Class Structure

Our custom `Fruits360Dataset` class integrates all the above methodologies into a unified pipeline compatible with PyTorch.

5.6.1 The `__getitem__` Method

For every index called by the DataLoader, the following sequence occurs: 1. **Image Loading:** Read JPG file, convert to RGB (for output) and Grayscale (for processing). 2. **Feature Extraction:** Calls helper functions to get the 6 Color stats and 6 Shape stats. These are concatenated into a single Tensor of size 12. 3. **Processing:** Generates the LBP and Canny images. These are normalized to $[0, 1]$ and stacked into a Tensor of size $(2, 100, 100)$. 4. **Normalization:** The original RGB image is permuted to channel-first format $(3, 100, 100)$ and normalized.

5.6.2 Output Format

The final data packet yielded for training is a dictionary containing:

- `'original'`: Raw visual data (RGB).
- `'processed'`: Engineered visual data (Texture + Edges).
- `'features'`: Engineered numerical data (Color + Shape).
- `'label'`: The classification target.

Lecture 6

Mathematical Toolkit for Machine Learning

6.1 Module 1: Linear Regression - The Mathematics of Prediction

Linear regression aims to model a continuous relationship between input features and a target variable.

6.1.1 Core Concept and Hypothesis

The fundamental goal is to model a linear relationship between input features x and a continuous target variable y. The model's prediction, $\hat{y}$, is a linear combination of input features, represented in vectorized form as:

$$\hat{y} = h_w(x) = w^T x$$

where w is the D-dimensional vector of model weights (parameters) and x is the D-dimensional feature vector.

6.1.2 Objective Function: Mean Squared Error (MSE)

To find the optimal weights w, we minimize the difference between predictions $\hat{y}$ and true values y. The objective is to minimize the Mean Squared Error (MSE) loss function, $L(w)$:

$$L(w) = \frac{1}{N} \sum_{n=1}^{N} (y_n - w^T x_n)^2$$

where N is the number of training examples.

6.1.3 Optimization

We can optimize the weights using two primary methods:

Gradient Descent

We iteratively adjust weights in the direction opposite to the gradient:

$$w(t+1) = w(t) - \eta \cdot \nabla_w L(w(t))$$

The update rule becomes:

$$w(t+1) = w(t) + \eta \frac{2}{N} \sum_{n=1}^{N} (y_n - w(t)^T x_n) \cdot x_n$$

Normal Equation (Closed-Form)

For smaller datasets, we can solve directly by setting the gradient to zero:

$$w = (X^T X)^{-1} X^T y$$

While direct, this method requires computing the inverse of $(X^T X)$, which is computationally prohibitive for large dimensions D.

6.1.4 Evaluation

Performance is measured using the Coefficient of Determination (R^2):

$$R^2 = 1 - \frac{\sum (y_n - \hat{y}_n)^2}{\sum (y_n - \overline{y})^2}$$

An R^2 of 1 indicates a perfect fit, while 0 indicates the model performs no better than predicting the mean.

6.2 Module 2: Logistic Regression - The Mathematics of Classification

Logistic Regression is used for binary classification problems where $y \in \{0, 1\}$.

6.2.1 Formulation

A linear score $z = w^T x$ is passed through the sigmoid function $\sigma(z)$ to map it to a probability between 0 and 1:

$$\hat{y} = P(y = 1|x; w) = \sigma(w^T x) = \frac{1}{1 + e^{-w^T x}}$$

6.2.2 Objective Function: Binary Cross-Entropy (BCE)

The goal is to minimize the Negative Log-Likelihood. The Binary Cross-Entropy loss is defined as:

$$L(w) = -\frac{1}{N} \sum_{n=1}^{N} [y_n \cdot \log(\hat{y}_n) + (1 - y_n) \cdot \log(1 - \hat{y}_n)]$$

6.2.3 Multi-Class Extension (Softmax)

For $C > 2$ classes, the Softmax function is used:

$$P(y = i|x; W) = \frac{\exp(w_i^T x)}{\sum_{j=1}^{C} \exp(w_j^T x)}$$

6.2.4 Evaluation Metrics

Unlike regression, classification is evaluated using a Confusion Matrix. Key metrics include:

- **Accuracy:** $\frac{TP+TN}{TP+TN+FP+FN}$
- **Precision:** $\frac{TP}{TP+FP}$
- **Recall (Sensitivity):** $\frac{TP}{TP+FN}$
- **AUC-ROC:** Measures performance across all classification thresholds.

6.3 Module 3: K-Means Clustering - The Mathematics of Grouping

K-Means is an unsupervised algorithm that partitions a dataset into K distinct, non-overlapping clusters.

6.3.1 Objective Function

The algorithm minimizes the Within-Cluster Sum of Squares (WCSS), also called inertia:

$$J = \sum_{k=1}^{K} \sum_{x_n \in C_k} ||x_n - \mu_k||^2$$

where μ_k is the centroid of cluster C_k.

6.3.2 Algorithm (Lloyd's Algorithm)

The process involves two iterative steps:

1. **Assignment Step:** Assign each data point to the closest centroid.
2. **Update Step:** Recalculate centroids as the mean of assigned points:

$$\mu_k^{(t)} = \frac{1}{|C_k^{(t)}|} \sum_{x_n \in C_k^{(t)}} x_n$$

6.3.3 Initialization and Evaluation

- **K-Means++:** A smarter initialization strategy that selects initial centroids probabilistically proportional to their squared distance from existing centroids.
- **Elbow Method:** Plots WCSS vs. K to find the point of diminishing returns.
- **Silhouette Score:** Measures how similar a point is to its own cluster compared to others, ranging from -1 to 1.

6.4 Module 4: Multilayer Perceptron (MLP) - The Mathematics of Non-Linearity

An MLP is a feedforward neural network capable of learning complex, non-linear functions.

6.4.1 Mathematical Formulation

Each neuron computes a weighted sum of inputs, adds a bias, and applies a non-linear activation function $f(\cdot)$:

$$a = f(z) = f\left(\sum_{i=1}^{D} w_i x_i + b\right)$$

6.4.2 Common Activation Functions

- **Sigmoid:** $\sigma(z) = \frac{1}{1+e^{-z}}$
- **Tanh:** $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$
- **ReLU:** $f(z) = \max(0, z)$

6.4.3 Backpropagation

Backpropagation efficiently computes gradients of the loss function using the chain rule, moving backward from the output layer:

$$\frac{\partial L}{\partial w^{(l)}} = \frac{\partial L}{\partial a^{(L)}} \cdot \dots \cdot \frac{\partial a^{(l)}}{\partial z^{(l)}} \cdot \frac{\partial z^{(l)}}{\partial w^{(l)}}$$

This allows optimization via Gradient Descent.

6.5 Module 5: Convolutional Neural Networks (CNN) - The Mathematics of Perception

CNNs are specialized for grid-like topology data (images), using layers to learn hierarchical spatial features.

6.5.1 Core Building Blocks

- **Convolutional Layer:** Applies learnable filters (kernels) to extract features.
- **Pooling Layer:** Reduces spatial dimensions (subsampling).
- **Fully Connected Layer:** Performs final classification.

6.5.2 The Convolution Operation

The core operation is a 2D discrete convolution between input I and kernel K. The output feature map G is:

$$G[i,j] = (I * K)[i,j] = \sum_m \sum_n I[i-m, j-n] \cdot K[m,n]$$

6.5.3 Pooling Operation

Pooling summarizes a neighborhood R. For Max Pooling:

$$y = \max(x_i) \quad \text{for } x_i \in R$$

This provides robustness to small translations (Translation Invariance).

6.5.4 Key Principles

- **Parameter Sharing:** The same kernel is applied across the entire image, drastically reducing parameters.
- **Translation Invariance:** Features can be detected regardless of their position in the image.

Lecture 7

Model Generalization, Regularization and Neural Networks

7.1 Model Generalization

Model generalization refers to the ability of a learning algorithm to perform accurately on unseen data. Given a dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$, a model learns a function $f(x;\theta)$ parameterized by θ. The goal is not only to minimize error on training data, but also to minimize the expected risk:

$$\mathbb{E}_{(x,y)\sim P_{\text{data}}}\left[(y - f(x;\theta))^2\right].$$

Poor generalization results in underfitting or overfitting.

7.2 Underfitting

Underfitting occurs when the hypothesis class is too simple to model the underlying data distribution. Consider a linear regression model $f(x) = wx + b$. If the true data relationship is non-linear (e.g., $y = x^2+\epsilon$), a linear model will fail to capture the curvature, resulting in large errors.

7.2.1 Error Characteristics

The Mean Squared Error (MSE) is defined as $\text{MSE} = \frac{1}{N}\sum_{i=1}^N (y_i - \hat{y}_i)^2$. In underfitting, both $\text{MSE}_{\text{train}}$ and MSE_{test} are high, indicating high bias and insufficient model capacity.

7.3 Overfitting

Overfitting occurs when the model is overly complex and learns noise in the training data. Consider a high-degree polynomial model. As the degree increases, the model fits training data extremely well but becomes sensitive to small variations.

7.3.1 Error Characteristics

In overfitting, $\text{MSE}_{\text{train}} \approx 0$ while MSE_{test} is high. This indicates high variance and poor generalization.

7.4 Bias–Variance Trade-off

The expected prediction error can be decomposed as:

$$\mathbb{E}[(y - \hat{f}(x))^2] = \text{Bias}^2 + \text{Variance} + \text{Noise}.$$

- High bias $\Rightarrow$ underfitting
- High variance $\Rightarrow$ overfitting

An optimal model balances bias and variance.

7.5 Regularization

Regularization reduces overfitting by constraining model parameters. The loss function is modified as:

$$\mathcal{L}_{\text{reg}} = \mathcal{L}_{\text{data}} + \lambda \Omega(\mathbf{w}),$$

where λ controls the regularization strength.

7.5.1 L2 Regularization (Ridge)

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2 + \lambda \sum_j w_j^2$$

L2 regularization penalizes large weights and keeps parameters small, resulting in smoother models.

7.5.2 L1 Regularization (Lasso)

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2 + \lambda \sum_j |w_j|$$

L1 regularization promotes sparsity by forcing some weights to exactly zero, effectively performing feature selection.

7.6 Feature Scaling

Feature scaling ensures numerical stability and faster convergence during optimization. Standardization is defined as $x' = \frac{x-\mu}{\sigma}$. Without scaling, gradients may oscillate, slowing convergence.

7.7 Multilayer Perceptron (MLP)

An MLP consists of multiple layers of neurons. For a single neuron, $z = \sum_i w_i x_i + b$ and $a = \phi(z)$, where $\phi(\cdot)$ is an activation function such as ReLU. For a hidden layer, $\mathbf{h} = \phi(\mathbf{W}\mathbf{x} + \mathbf{b})$. MLPs enable learning of complex non-linear mappings.

7.8 Universal Approximation Theorem

The theorem states that a feedforward neural network with at least one hidden layer and non-linear activation functions can approximate any continuous function to arbitrary precision given sufficient neurons.

7.9 Backpropagation

Backpropagation computes gradients of the loss function using the chain rule. Given loss $\mathcal{L} = \frac{1}{2}(y - \hat{y})^2$, the gradient w.r.t. weight w is:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial w}.$$

Using gradient descent, the weight update rule is $w^{(t+1)} = w^{(t)} - \eta \frac{\partial \mathcal{L}}{\partial w}$.

Lecture 8

Convolutional Neural Networks and Model Compression

8.1 Introduction to Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are a class of deep neural networks designed specifically for image and spatial data. Unlike traditional fully connected neural networks, CNNs exploit the spatial structure of images by using local connectivity and shared weights. This significantly reduces the number of parameters and improves computational efficiency. An image of size $H \times W$ with C channels is represented as a tensor $\mathbf{X} \in \mathbb{R}^{H \times W \times C}$.

8.2 Limitations of Fully Connected Networks for Images

In a fully connected network, an image of size 100×100 would result in 10,000 input neurons. Connecting this to even a single hidden layer leads to an extremely large number of weights ($10{,}000 \times N$), which is computationally expensive and prone to overfitting. CNNs overcome this problem by using convolutional filters that operate locally.

8.3 Convolution Operation

The convolution operation applies a small filter (kernel) across the input image to extract local features such as edges and textures. For a 2D convolution:

$$Y(i,j) = \sum_m \sum_n X(i+m, j+n)\, K(m,n).$$

This operation allows the network to learn spatial patterns efficiently.

8.4 Weight Sharing and Translational Invariance

In CNNs, the same filter is applied across the entire image. This concept is known as weight sharing and drastically reduces the number of learnable parameters. Because

the same filter detects a feature regardless of its position, CNNs exhibit translational invariance.

8.5 Activation Function

After convolution, a non-linear activation function is applied. The most commonly used activation in CNNs is the Rectified Linear Unit (ReLU): $\text{ReLU}(x) = \max(0, x)$. ReLU introduces non-linearity and helps mitigate the vanishing gradient problem.

8.6 Pooling Operation

Pooling reduces the spatial dimensions of feature maps while retaining important information.

8.6.1 Max Pooling

For a pooling window of size $k \times k$:

$$Y(i, j) = \max_{m,n \in k \times k} X(i + m, j + n).$$

Pooling improves computational efficiency and provides robustness to small translations in the input.

8.7 CNN Architecture

A typical CNN architecture consists of: Convolution layer, Activation function, Pooling layer, and Fully connected layers. The final layer often uses the softmax function for classification: $\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$.

8.8 Model Compression

Deep neural networks often require large memory and computational resources. Model compression techniques are used to reduce model size and inference time.

8.8.1 Pruning

Pruning removes less important weights from the network. If w_i represents a weight, pruning applies a threshold:

$$w_i = 0 \quad \text{if } |w_i| < \epsilon.$$

This reduces memory usage without significantly affecting performance.

8.9 Quantization

Quantization reduces the precision of weights from 32-bit floating point to lower-bit representations.

8.9.1 Memory Reduction

For an $n \times n$ weight matrix:

$$\text{Original memory} = n^2 \times 32 \text{ bits}, \quad \text{Quantized memory} = n^2 \times 8 \text{ bits}.$$

This leads to a $4\times$ reduction in memory usage.

8.10 Clustering-Based Quantization

Weights are grouped using clustering (e.g., K-means). Each weight is replaced by its cluster centroid. The centroid is computed as $\mu = \frac{1}{N}\sum_{i=1}^{N} w_i$. During backpropagation, gradients are accumulated for cluster means instead of individual weights.

8.11 Gradient Flow in Quantized Networks

Using the chain rule: $\frac{dL}{dw} = \frac{dL}{d\mu} \cdot \frac{d\mu}{dw}$. Since $\mu = \frac{1}{N}\sum w_i \Rightarrow \frac{d\mu}{dw} = \frac{1}{N}$, the constant factor is absorbed into the learning rate.

Lecture 9

Deep Learning Implementation with PyTorch

9.1 Introduction

This chapter shifts focus from mathematical theory to the practical implementation of Deep Learning using **PyTorch**. While TensorFlow remains a competitor, PyTorch has become the dominant framework in research (used in $\approx$ 92% of 2024/25 papers) and Generative AI due to its "Python-first" philosophy and dynamic computation graphs.

9.2 The Fundamental Unit: Tensors

A **Tensor** is a multi-dimensional array specialized for GPU acceleration. It generalizes scalars, vectors, and matrices:

- **0-D (Scalar):** A single number (e.g., loss value).
- **1-D (Vector):** A list of numbers (e.g., bias vector).
- **2-D (Matrix):** A grid of numbers (e.g., grayscale image).
- **N-D Tensor:** Higher-order structures (e.g., a batch of RGB images: $B \times C \times H \times W$).

9.2.1 Creation and Operations

Tensors can be created from standard Python lists or NumPy arrays. Key operations include:

- **Shape Manipulation:** `view()` or `reshape()` to change dimensions without altering data.
- **Device Management:** `.to('cuda')` moves tensors to the GPU for parallel processing.

9.3 Data Handling Pipeline

PyTorch decouples data processing from model training using two primitives:

1. **Dataset:** Stores samples and their corresponding labels. It implements `__len__` and `__getitem__`.
2. **DataLoader:** Wraps an iterable around the Dataset to handle batching, shuffling, and parallel loading.

9.4 Autograd: The Automatic Differentiation Engine

The core feature distinguishing PyTorch from NumPy is **Autograd**. It automatically computes gradients for backpropagation.

9.4.1 Computational Graph

PyTorch constructs a dynamic Directed Acyclic Graph (DAG) where nodes represent Tensors and edges represent mathematical operations. When `loss.backward()` is called, the engine traverses this graph from the output to the inputs using the Chain Rule to populate the `.grad` attribute.

9.4.2 Gradient Control

- `requires_grad=True`: Signals PyTorch to track operations on this tensor.
- `torch.no_grad()`: A context manager that disables gradient tracking, used during inference to reduce memory usage.

9.5 The Training Workflow

A standard PyTorch training loop consists of:

1. **Forward Pass:** Compute predictions $\hat{y}$ from inputs x.
2. **Loss Calculation:** Compute error $J(\theta)$.
3. **Backward Pass:** Call `loss.backward()` to compute gradients $\nabla_\theta J$.
4. **Optimizer Step:** Update parameters $\theta \leftarrow \theta - \alpha \nabla_\theta J$.
5. **Zero Gradients:** Reset gradients to zero before the next iteration.